

Primera entrega - API Rest

Seminario de Lenguajes Opción: PHP, React y API Rest 2026

API REST / Slim

Slim es un micro framework de PHP que se utiliza para crear aplicaciones web y APIs REST. Las API REST siguen el modelo CRUD, que significa Create (Crear), Retrieve (Recuperar), Update (Actualizar) y Delete (Eliminar). Los recursos de una API REST se identifican mediante endpoints o URI (Uniform Resource Identifier). Cada recurso tiene una dirección única a la que se puede acceder para realizar operaciones sobre él. Generalmente, se utiliza JSON (JavaScript Object Notation) como formato para el intercambio de datos entre el cliente y el servidor.

ACLARACIONES

Se deben tener las siguientes consideraciones:

- Todos los endpoints deberán tener los métodos y nombres que se detallan en cada ítem.
- Las eliminaciones sólo se podrán ejecutar correctamente en los casos en que el dato no esté siendo utilizado en otra tabla, caso contrario se debe informar el error.

IMPORTANTE

En todos los casos, los endpoints deberán devolver el estado de la petición realizada:

- **200 OK:** Indica que la solicitud se procesó correctamente y se devolvió una respuesta exitosa.
- **400 Bad Request:** Se utiliza cuando la solicitud enviada por el cliente es incorrecta o no se puede procesar debido a parámetros faltantes, valores inválidos o problemas de formato.
- **401 Unauthorized:** Indica que un usuario no puede realizar la acción en cuestión porque no está autorizado.
- **404 Not Found:** Indica que el recurso solicitado no se pudo encontrar en el servidor.
- **409 Conflict:** Indica que el registro no pudo ser eliminado o que la operación compra/venta no se pudo realizar.

En los casos en que se produce un error, además, deberán devolver un mensaje indicando por qué no se pudo realizar la acción.

Nota: Los mensajes de error deben ser autoexplicativos, el usuario debe recibir la información necesaria para solucionarlo (si puede).

PROYECTO: SIMULADOR DE INVERSIONES "WALLY STREET"

Bienvenidos al nuevo sistema de inversiones de la facultad. El objetivo de este proyecto es desarrollar una aplicación web funcional que simule un entorno de mercado financiero. Los

usuarios registrados podrán gestionar un capital inicial e invertir en activos volátiles para intentar incrementar su patrimonio.

Al comenzar, cada nuevo usuario que se registre en el sistema recibirá un bono único de bienvenida de **1000 USD** de saldo virtual. Con este capital, el inversor podrá acceder a una pizarra de cotizaciones en tiempo real que ofrece siete activos específicos: **Oro, Plata, YPF, Petróleo, Bitcoin, Apple y Soja.**

El sistema debe permitir que el usuario realice operaciones de compra y venta de estos activos. Al comprar, el sistema descontará el monto total (precio actual por cantidad de unidades) del saldo en efectivo del usuario y añadirá dichas unidades a su inventario personal o "portfolio". Al vender, el sistema calculará el valor de las unidades según el precio vigente en ese momento, sumará el dinero obtenido al saldo del usuario y restará las unidades de su inventario. Es requisito indispensable que el sistema valide que el usuario cuenta con saldo suficiente para comprar o con unidades suficientes para vender, impidiendo operaciones que generen saldos negativos.

Un componente central de la aplicación es la actualización de precios. Periódicamente, los valores de los siete activos deben variar de forma aleatoria, simulando la volatilidad del mercado. El usuario deberá estar atento a estos cambios para decidir si mantiene sus activos, compra más o vende para asegurar ganancias.

Finalmente, la aplicación debe contar con una sección de historial donde se registren detalladamente todas las transacciones realizadas, indicando la fecha, el tipo de operación (compra o venta), el activo involucrado, la cantidad de unidades y el precio al que se cerró la negociación.

1. Crear una función `variarPrecioPorTiempo` que calcule la variación de un asset. A continuación, se presenta un ejemplo de una posible implementación.

```
function variarPrecioPorTiempo($precioActual, $timestampUltimaVez,
$volatilidadPorSegundo = 0.05) {

// 1. Calcular cuántos segundos han pasado

$tiempoPasado = time() - $timestampUltimaVez; // Si no ha pasado tiempo, el precio no
cambia

if ($tiempoPasado <= 0) return $precioActual;

// 2. Generar un cambio aleatorio (puede ser positivo o negativo)

// mt_rand(-100, 100) / 100 nos da un número entre -1.0 y 1.0

$direccion = mt_rand(-100, 100) / 100;

// 3. El cambio total depende del tiempo que pasó
```

```

$delta = $direccion * $volatilidadPorSegundo * $tiempoPasado;

return $precioActual + $delta;

}

```

Ejemplo:

```

$precioAnterior = 100; //Buscar ultima cotización del asset

$actualizadoHace = time() - 10;

// Simulamos que pasaron 10 segundos

$nuevoPrecio = variarPrecioPorTiempo($precioAnterior, $actualizadoHace);

echo "Precio viejo: " . $precioAnterior . "\n";

echo "Pasaron 10 segundos...\n";

echo "Precio nuevo: " . round($nuevoPrecio, 2);

```

2. Implementar los siguientes endpoints.

AUTENTICACIÓN

Se mantiene el mecanismo anterior:

- **POST /login:** Recibe email/password. Genera un token aleatorio con **expired** a 5 minutos.
- **POST /logout:** Borra el token y el expired correspondiente de la tabla de usuario.
- **Middleware:** Cada interacción autorizada (**Authorization: Bearer <token>**) estira la vida del token 5 minutos más.

USUARIOS

- **POST /users:** Registro de nuevo inversor.
 - Validaciones
 - email (@) y password (minúscula, mayúscula, número, especial, 8 caracteres).
 - email debe ser único.
 - name: Solo letras. No puede ser vacío.
 - **Acción automática:** Al crearse, el usuario recibe el bono de **1000 USD** en su campo **balance**.
- **GET /users/{user_id}:** Ver perfil, saldo actual y valor total del portfolio.
 - *Requiere que user_id sea el usuario logueado o que el usuario logueado sea admin.*

- **PUT /users/{user_id}**: Editar un usuario existente. Requiere que user_id sea el usuario logueado. Se deben enviar **solo** los datos a modificar.
 - *Solo se puede modificar el name y/o password*
 - *Requiere que user_id sea el usuario logueado o que el usuario logueado sea admin.*
 - **GET /users**: (Solo admin) Listar inversores para monitoreo. Solo nombre y valor total del portfolio.
-

ACTIVOS (El Mercado)

- **GET /assets**: Lista activos (Oro, Plata, YPF, etc.) con su precio actual de acuerdo a los parámetros de búsqueda. Si no hay parámetros debe devolver todos los registros. No requiere login.
 - Filtros opcionales:
 - `?type={name}`
 - `min_price`: Precio mínimo (ej: `?min_price=50`).
 - `max_price`: Precio máximo (ej: `?max_price=500`).
 -
 - **PUT /assets**: (Solo Admin) Dispara la actualización aleatoria de los precios de todos los activos.
 - **GET /assets/{asset_id}/history/{quantity}**: Muestra los últimos cambios de precio de un activo específico (máximo 5 últimos movimientos).
 - *No requiere login*
 - Usar la tabla transactions sin revelar el usuario que hizo la transacción.
-

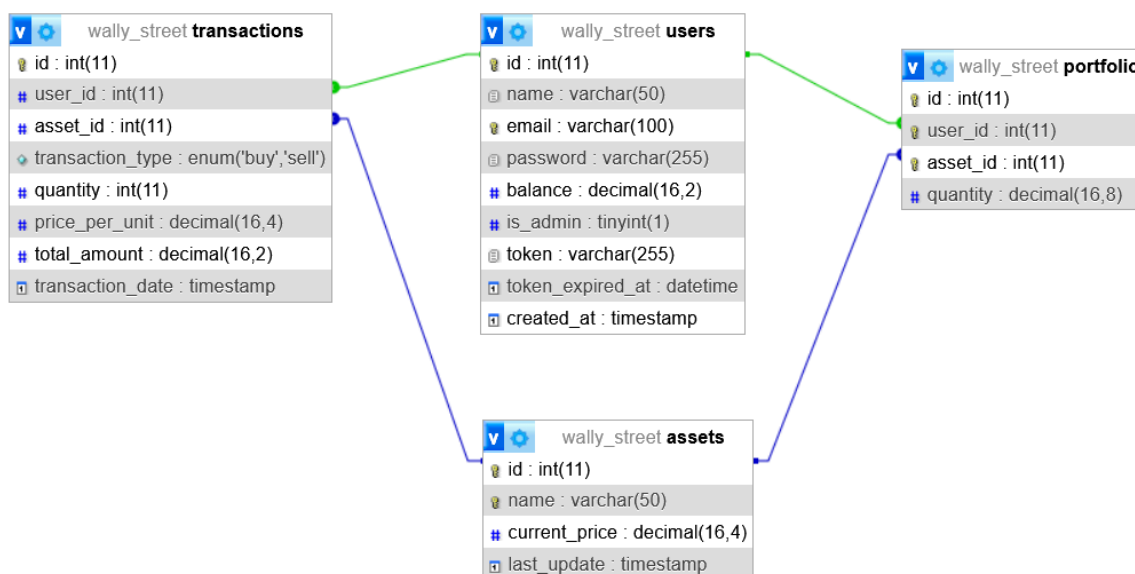
OPERACIONES (Wally Street Core)

- **POST /trade/buy**: Comprar un activo.
 - **Requisito**: El usuario debe estar logueado.
 - **Cuerpo**: `asset_id`, `quantity`.
 - **Validaciones**:
 1. El usuario debe tener `balance >= (current_price * quantity)`.
 2. El `asset_id` debe existir.
 - **Resultado**: Resta saldo al usuario, suma unidades en la tabla `portfolio` y registra en `transactions`.
- **POST /trade/sell**: Vender un activo.
 - **Requisito**: El usuario debe estar logueado.
 - **Cuerpo**: `asset_id`, `quantity`.
 - **Validaciones**:
 1. El usuario debe poseer en su `portfolio` una cantidad `>= quantity` de ese activo.

- **Resultado:** Suma saldo al usuario ($\text{current_price} * \text{quantity}$), resta unidades del **portfolio** y registra en **transactions**.

PORTFOLIO e HISTORIAL

- **GET /portfolio:** Lista los activos que posee el usuario logueado (ej: "Bitcoin: 0.5, YPF: 10"). Debe calcular el valor actual de esa tenencia según los precios de mercado.
 - **Requisito:** El usuario debe estar logueado.
- **DELETE /portfolio/{asset_id}:** Eliminar un activo del inventario personal.
 - **Requisito:** El usuario debe estar logueado.
 - **Validación Crítica:** Solo se puede borrar el registro si la **cantidad** es exactamente **0**.
 - **Lógica de error:**
 - Si el usuario tiene **1** de Bitcoin o más, el endpoint devuelve **409 Conflict** con el mensaje: *"No puedes quitar un activo de tu portfolio si aún tienes unidades. Debes venderlas primero."*
 - Si el activo no existe en su portfolio, devuelve **404 Not Found**.
- **Resultado:** Código **200 OK**. Se borra el asset indicado de la tabla portfolio del usuario.
- **GET /transactions:** Devuelve el historial de movimientos del usuario ordenado por fecha descendente.
 - Filtros opcionales: **?type=buy** o **?type=sell** y/o **?asset_id=x**.
 - **Requisito:** El usuario debe estar logueado.



METODOLOGÍA PARA LA ENTREGA EN IDEAS

- Hacer un archivo zip o rar con todos los archivos del proyecto sin incluir la carpeta vendor. Si se utilizan librerías externas agregar un README indicando para qué las utilizaron y su instalación
- Subir a la sección del grupo asignada (En el repositorio general NO van)
- Fecha de entrega: 11/05

-- 1. Creación de la Base de Datos

```
CREATE DATABASE IF NOT EXISTS seminariophp;
```

```
USE seminariophp;
```

-- 2. Tabla de Usuarios

```
CREATE TABLE users ( id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50) NOT NULL, email VARCHAR(100) NOT NULL UNIQUE, password VARCHAR(255) NOT NULL, balance DECIMAL(16, 2) DEFAULT 1000.00, is_admin TINYINT(1) DEFAULT 0, token VARCHAR(500) NULL, token_expired_at DATETIME NULL, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

-- 3. Tabla de Activos (Assets)

```
CREATE TABLE assets ( id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50) NOT NULL UNIQUE, current_price DECIMAL(16, 2) NOT NULL, last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP );
```

-- 4. Tabla de Portfolio (Relaciona usuarios con sus activos)

```
CREATE TABLE portfolio ( id INT AUTO_INCREMENT PRIMARY KEY, user_id INT NOT NULL, asset_id INT NOT NULL, quantity INT NOT NULL DEFAULT 0, FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE, FOREIGN KEY (asset_id) REFERENCES assets(id), UNIQUE KEY unique_user_asset (user_id, asset_id) );
```

-- 5. Tabla de Transacciones (Historial)

```
CREATE TABLE transactions ( id INT AUTO_INCREMENT PRIMARY KEY, user_id INT NOT NULL, asset_id INT NOT NULL, transaction_type ENUM('buy', 'sell') NOT NULL, quantity INT NOT NULL, price_per_unit DECIMAL(16, 2) NOT NULL, total_amount DECIMAL(16, 2) NOT NULL, transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (user_id) REFERENCES users(id), FOREIGN KEY (asset_id) REFERENCES assets(id) );
```

-- 6. Inserción de los 7 activos iniciales

```
INSERT INTO assets (name, current_price) VALUES ('Bitcoin', 65000.50), ('YPF', 25.30),  
('Gold', 2300.15), ('Silver', 28.45), ('Petroleum', 85.20), ('Apple', 175.10), ('Soybean', 430.00);
```